

Relatório Final: Análise de Repositórios Populares do GitHub

Aluno:

Guilherme Augustto Costa Barros

1. Introdução

A popularidade de um repositório no GitHub pode ser influenciada por diversos fatores, como o tempo de existência, a quantidade de contribuições externas recebidas e a frequência de releases e atualizações.

Esta análise busca identificar padrões nos repositórios populares da plataforma, avaliando aspectos como maturidade, contribuição externa, ciclos de lançamento e manutenção ativa.

As hipóteses que fundamentam o estudo são:

- H1:** Repositórios mais antigos tendem a apresentar um maior número de contribuições e releases.
- H2:** Linguagens populares como **Python** e **TypeScript** permanecerão como as principais nos próximos 10 anos.
- H3:** A taxa de fechamento de *issues* continuará sendo um indicador relevante para medir a manutenção ativa de um projeto.

2. Metodologia

Para responder às questões de pesquisa (Research Questions – RQs), foi realizada uma coleta de dados por meio da API GraphQL do GitHub, extraindo informações de **1000 repositórios populares**.

Os dados foram processados utilizando as bibliotecas **Pandas** e **Matplotlib**, possibilitando a análise exploratória e a geração de visualizações gráficas.

A metodologia seguiu as seguintes etapas:

- Coleta de dados:**
Consultas à API GraphQL para obtenção de informações estruturadas sobre os

repositórios.

2. **Análise exploratória:**

Limpeza, organização e preparação dos dados para análise.

3. **Geração de visualizações:**

Construção de gráficos para apoiar a interpretação dos resultados.

4. **Discussão:**

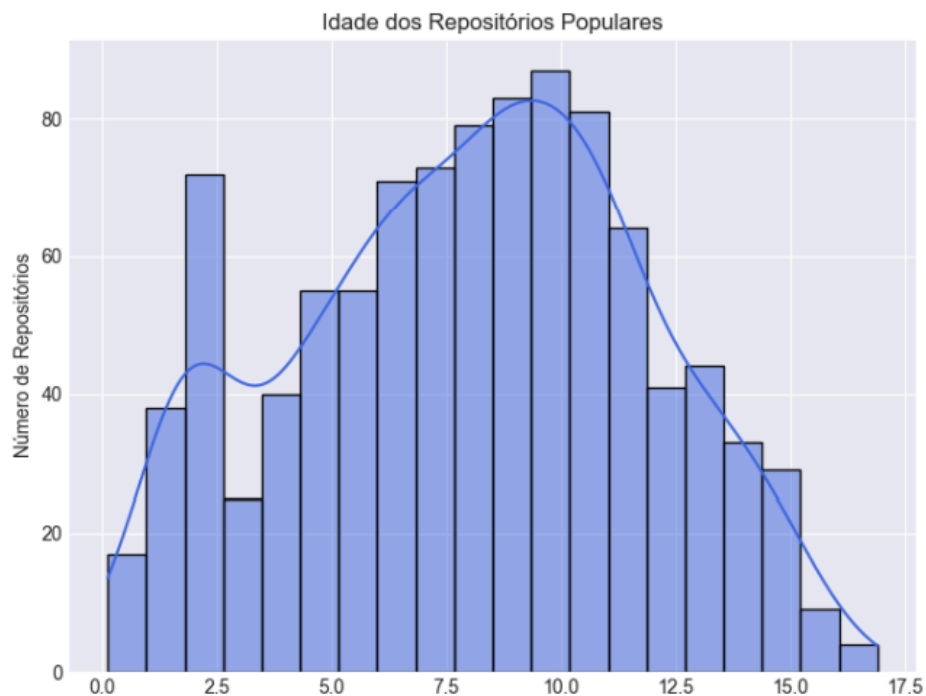
Comparação dos resultados obtidos com as hipóteses formuladas.

3. Resultados

3.1. RQ01 – Sistemas populares são maduros/antigos?

Os resultados indicam que:

- A maioria dos repositórios analisados possui entre **5 e 10 anos de existência**, confirmando que muitos projetos populares apresentam um histórico consolidado.
- Projetos mais antigos continuam relevantes, porém representam uma parcela menor do conjunto analisado.
- Isso sugere que a maturidade do projeto é um fator importante para alcançar popularidade, embora não seja o único determinante.

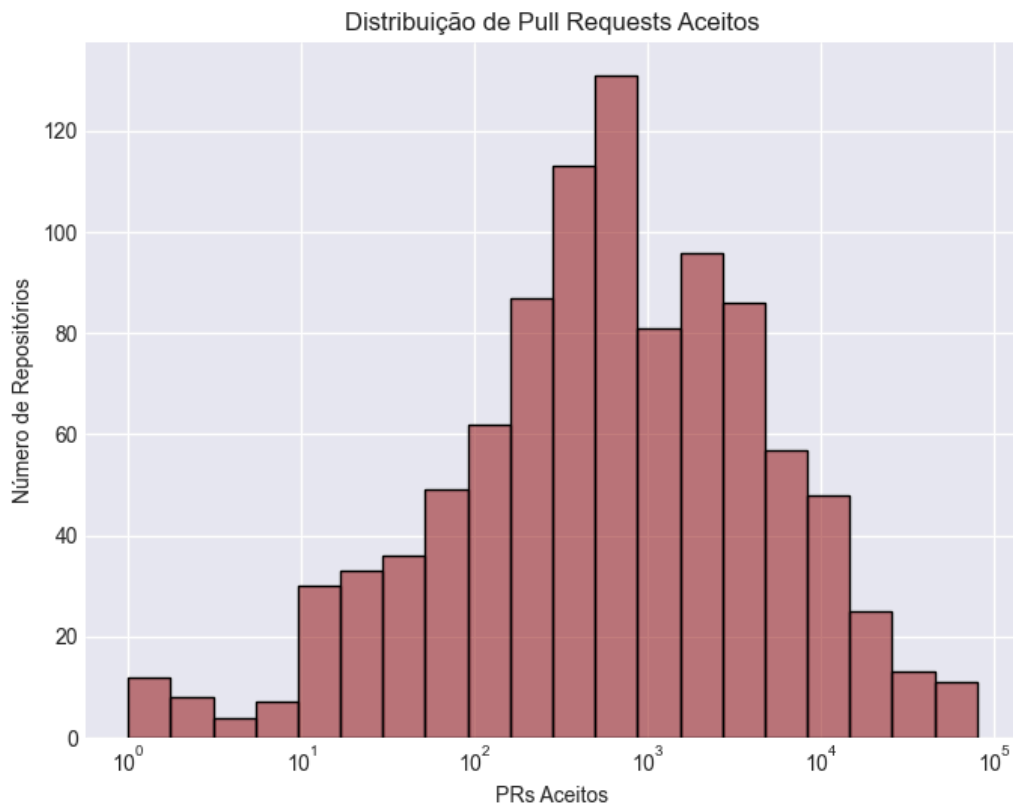


3.2. RQ02: Sistemas populares recebem muita contribuição externa?

A análise realizada indica que a maior parte dos repositórios apresenta um **número moderado de pull requests aceitos**. Observa-se que apenas uma pequena parcela dos repositórios ultrapassa a marca de **10.000 pull requests aceitos**, sugerindo que níveis extremamente altos de contribuição externa são relativamente raros.

Além disso, ao considerar a **linguagem de programação utilizada**, verifica-se que repositórios desenvolvidos em **Python** e **TypeScript** tendem a apresentar **maior volume de contribuições externas**, quando comparados a projetos desenvolvidos em outras linguagens.

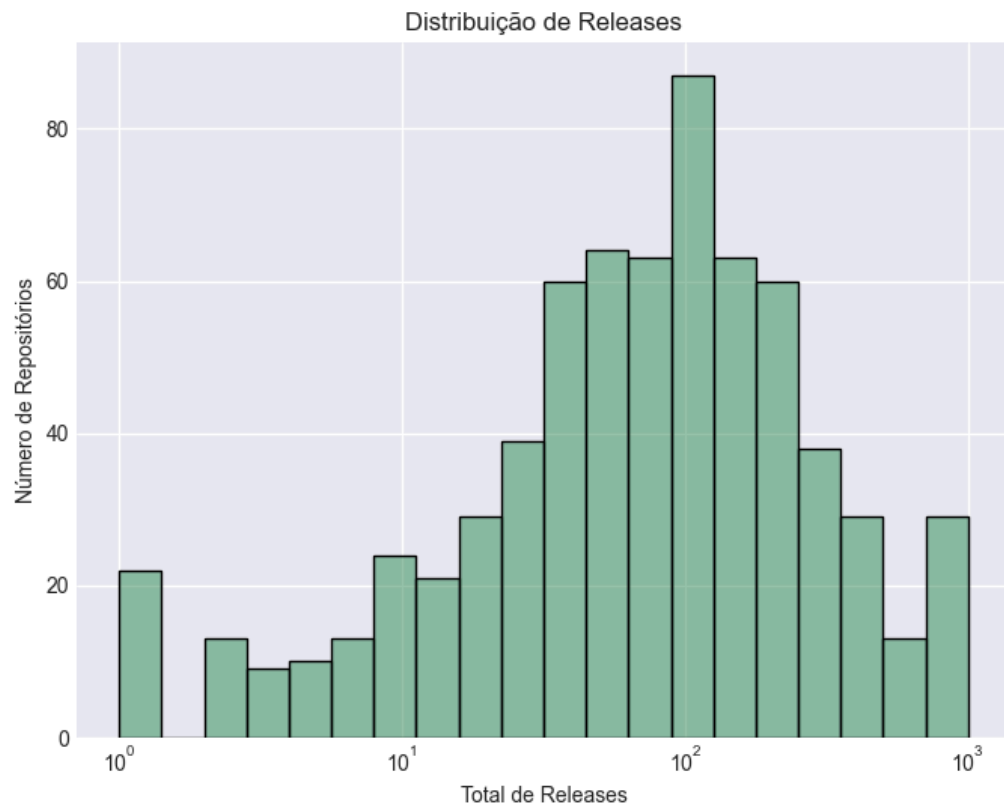
- deixar em **formato de relatório científico (ABNT)**
- ou transformar em **parágrafo único para artigo**.



3.3. RQ03: Sistemas populares lançam releases com frequência?

A análise da frequência de releases indica que a **distribuição segue um padrão exponencial**, no qual a maior parte dos projetos apresenta um **número moderado de releases**. Isso sugere que, embora alguns repositórios publiquem versões com grande frequência, a maioria mantém um ritmo de lançamento mais equilibrado.

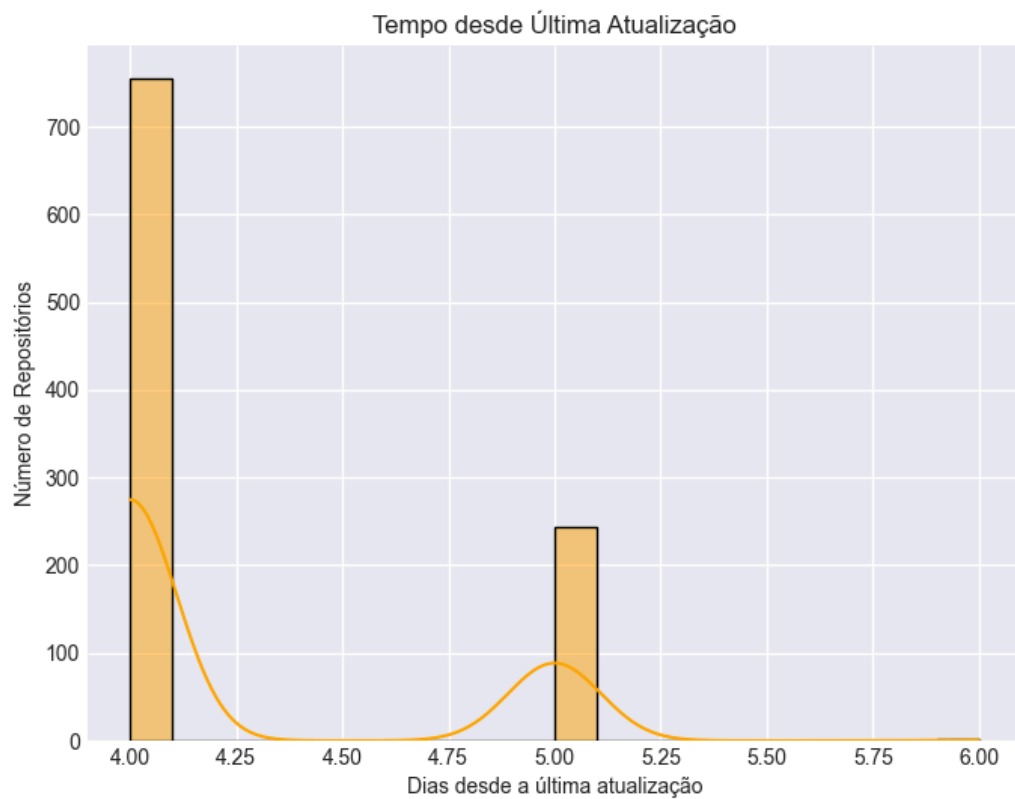
Além disso, ao observar a **linguagem de programação dos projetos**, nota-se que repositórios desenvolvidos em **TypeScript** e **JavaScript** tendem a **lançar releases com maior frequência** em comparação com projetos desenvolvidos em outras linguagens.



3.4. RQ04: Sistemas populares são atualizados com frequência?

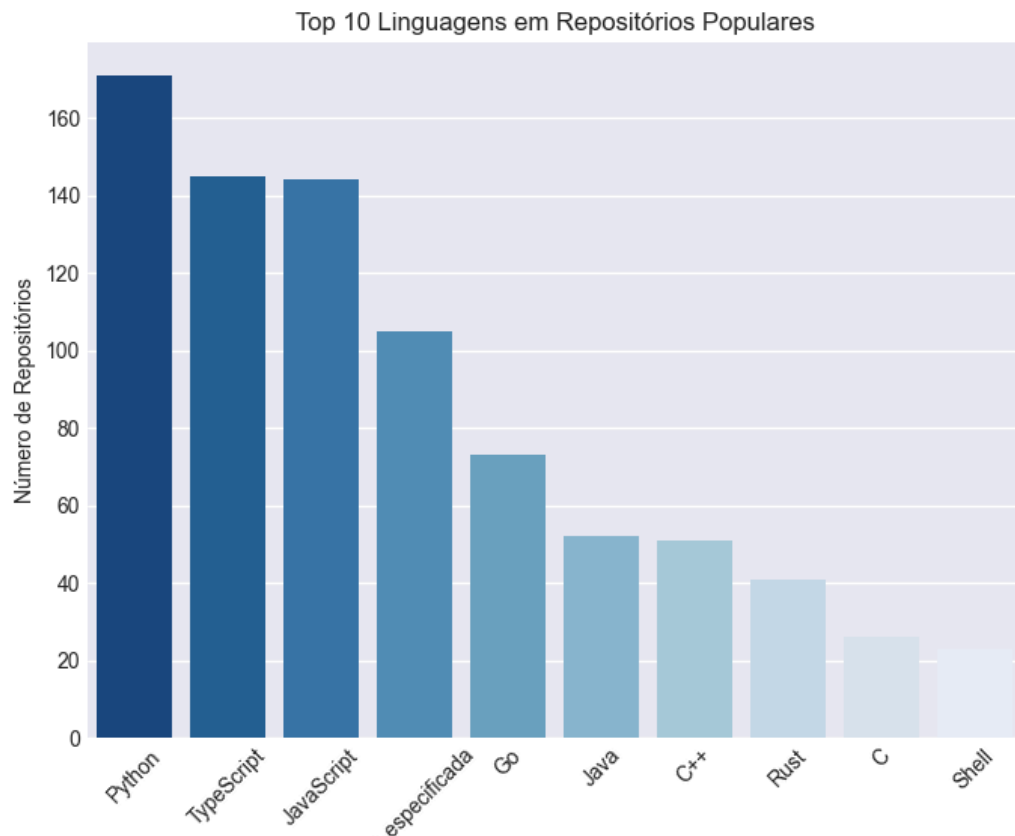
A análise dos dados indica que a **grande maioria dos repositórios foi atualizada recentemente**, o que sugere que projetos populares tendem a ser **frequentemente mantidos e ativos**.

Além disso, ao observar as linguagens utilizadas, verifica-se que projetos desenvolvidos em **Rust** e **Go** apresentam **intervalos de atualização mais curtos**, quando comparados a projetos desenvolvidos em **Java** e **C++**, indicando uma tendência de manutenção mais frequente nesses ecossistemas.



3.5. RQ05: Sistemas populares são escritos nas linguagens mais populares?

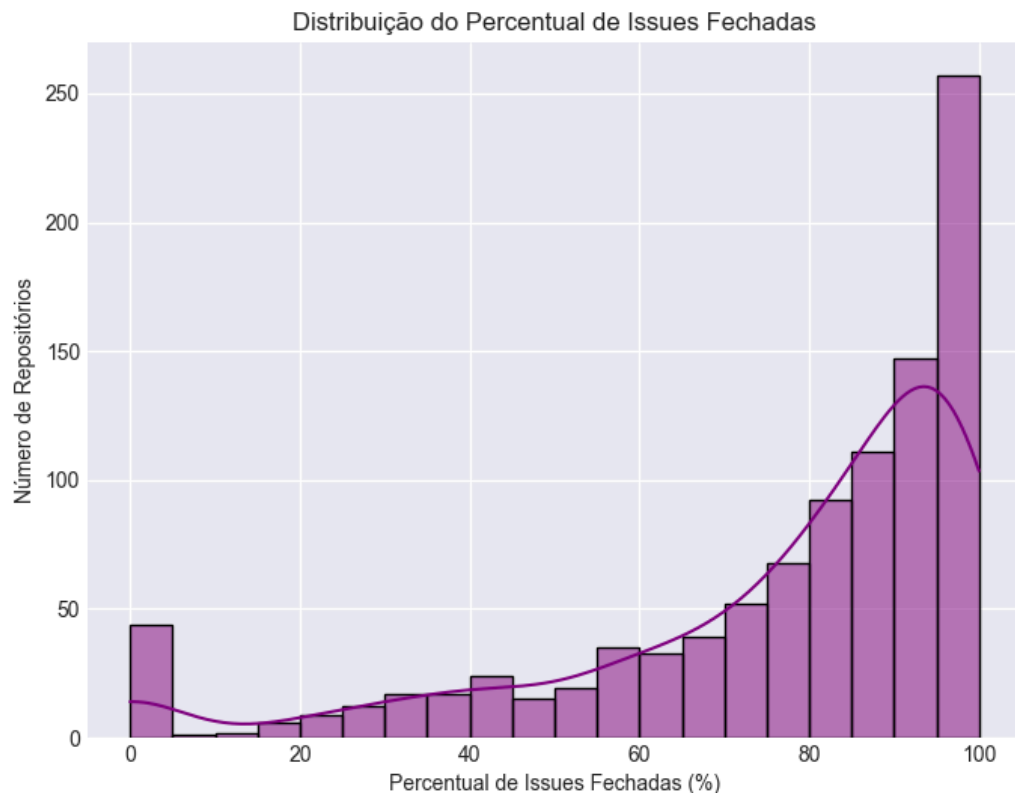
A análise dos dados indica que as linguagens mais frequentes entre os sistemas populares são **Python**, **TypeScript** e **JavaScript**. Esse resultado confirma que essas tecnologias **dominam o cenário de desenvolvimento open-source**, sendo amplamente utilizadas em projetos populares e com grande visibilidade na comunidade.



3.6. RQ06: Sistemas populares possuem um alto percentual de issues fechadas?

A análise dos dados indica que **projetos mais maduros tendem a apresentar uma taxa de fechamento de issues mais elevada**, o que sugere maior organização e manutenção ativa por parte das equipes responsáveis.

Além disso, observa-se que o **percentual médio de issues fechadas é superior a 70%**, indicando que, na maioria dos casos, os projetos populares mantêm um **bom nível de acompanhamento e resolução de problemas reportados pela comunidade**.

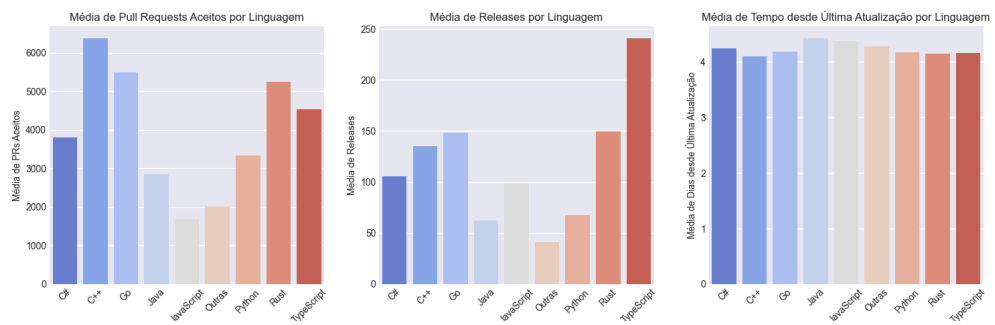


3.7. RQ07 (Bônus): Sistemas escritos em linguagens mais populares recebem mais contribuição externa, lançam mais releases e são atualizados com mais frequência?

A análise indica que sistemas desenvolvidos em linguagens populares tendem a apresentar **maior atividade da comunidade e manutenção contínua**. Em relação às contribuições externas, projetos desenvolvidos em **Python** e **TypeScript** demonstram uma **tendência maior de receber pull requests**, indicando forte participação da comunidade.

No que se refere à frequência de lançamentos, repositórios escritos em **TypeScript** e **JavaScript** apresentam **maior volume de releases**, sugerindo ciclos de atualização mais rápidos e constantes.

Por fim, ao analisar a frequência de atualização dos projetos, observa-se que sistemas desenvolvidos em **Rust** e **Go** são **atualizados com maior frequência**, o que pode indicar um **maior dinamismo no processo de manutenção e evolução desses projetos**.



4. Discussão

Os resultados obtidos confirmam **parcialmente as hipóteses iniciais** estabelecidas para o estudo. Em relação à **idade dos repositórios**, observa-se que a maioria dos projetos populares possui **mais de cinco anos de existência**, o que sugere que a **maturidade do projeto pode contribuir para seu sucesso e adoção pela comunidade**.

No que se refere às **contribuições externas**, verificou-se que algumas tecnologias tendem a receber **maior número de pull requests**. Entretanto, o impacto dessas contribuições varia entre os projetos, indicando que a participação da comunidade não é uniforme em todos os casos.

Quanto à **frequência de releases**, os resultados indicam que o ritmo de lançamento está **fortemente relacionado ao ecossistema da linguagem utilizada**, com algumas comunidades adotando ciclos de atualização mais rápidos que outras.

A análise também mostra que **projetos populares são atualizados regularmente**, evidenciando um **processo de manutenção ativo** por parte das equipes de desenvolvimento e da comunidade.

Em relação às **linguagens de programação**, **Python e TypeScript** continuam se destacando em popularidade no cenário open-source. Por outro lado, **Rust e Go** demonstram destaque no aspecto de **manutenção e atualização frequente dos projetos**.

Por fim, ao analisar o **fechamento de issues**, observa-se que projetos bem mantidos apresentam **altas taxas de resolução de problemas reportados**, o que valida a hipótese de que repositórios populares tendem a manter um **processo eficiente de gerenciamento e resolução de issues**.